

Python Automation Script

Objective

Create an automation script that performs file operations such as sorting, renaming, and cleaning files automatically.

Requirements Covered

- Use os module
- Use exception handling (try-except)
- Generate logs for operations
- Add user input support
- Perform file operations and sorting

Software Requirements

- Python 3.x
- VS Code / PyCharm / IDLE
- Windows / Linux / macOS

Project Description

This project organizes files in a selected folder into categories such as Images, Documents, Audio, Videos, Archives, and Others. It also creates a log file that records all operations and handles duplicate file names automatically.

Algorithm

1. Accept folder path from the user.
2. Validate the folder path.
3. Read all files.
4. Detect file types by extension.
5. Create category folders.
6. Move files.
7. Rename duplicates.
8. Generate logs.
9. Display success message.

Source Code

```
import os
import shutil
import logging
logging.basicConfig(
    filename='file_organizer.log',
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)
FILE_TYPES = {
    'Images': ['.jpg', '.jpeg', '.png', '.gif', '.bmp'],
    'Documents': ['.pdf', '.doc', '.docx', '.txt', '.ppt', '.pptx', '.xls', '.xlsx'],
```

```

'Audio': ['.mp3', '.wav', '.aac'],
'Videos': ['.mp4', '.mkv', '.avi', '.mov'],
'Archives': ['.zip', '.rar', '.7z'],
'Others': []
}
def get_category(filename):
    """
    Returns the category based on file extension.
    """
    _, ext = os.path.splitext(filename)
    ext = ext.lower()
    for category, extensions in FILE_TYPES.items():
        if ext in extensions:
            return category
    return 'Others'
def get_unique_destination(destination):
    """
    If a file with the same name already exists,
    create a new unique name like file_1.txt, file_2.txt.
    """
    if not os.path.exists(destination):
        return destination
    base, ext = os.path.splitext(destination)
    counter = 1
    while True:
        new_destination = f'{base}_{counter}{ext}'
        if not os.path.exists(new_destination):
            return new_destination
        counter += 1
def organize_files(folder_path):
    """
    Organizes files into category folders.
    """
    try:
        if not os.path.exists(folder_path):
            print("Error: Folder does not exist.")
            return
        if not os.path.isdir(folder_path):
            print("Error: The path is not a folder.")
            return
        print(f"\nOrganizing files in: {folder_path}")
        logging.info(f"Started organizing folder: {folder_path}")
        for item in os.listdir(folder_path):
            item_path = os.path.join(folder_path, item)
            if os.path.isdir(item_path) or item == 'file_organizer.log':
                continue
            category = get_category(item)
            category_folder = os.path.join(folder_path, category)
            os.makedirs(category_folder, exist_ok=True)
            destination = os.path.join(category_folder, item)
            destination = get_unique_destination(destination)
            shutil.move(item_path, destination)
            print(f"Moved: {item} -> {category}")
            logging.info(f"Moved {item} to {category}")
        print("\nOrganization completed successfully!")

```

```

        print("Log file created: file_organizer.log")
        logging.info("Organization completed successfully.")
    except PermissionError:
        print("Permission denied. Please run with proper access rights.")
        logging.exception("PermissionError occurred.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        logging.exception("Unexpected error occurred.")

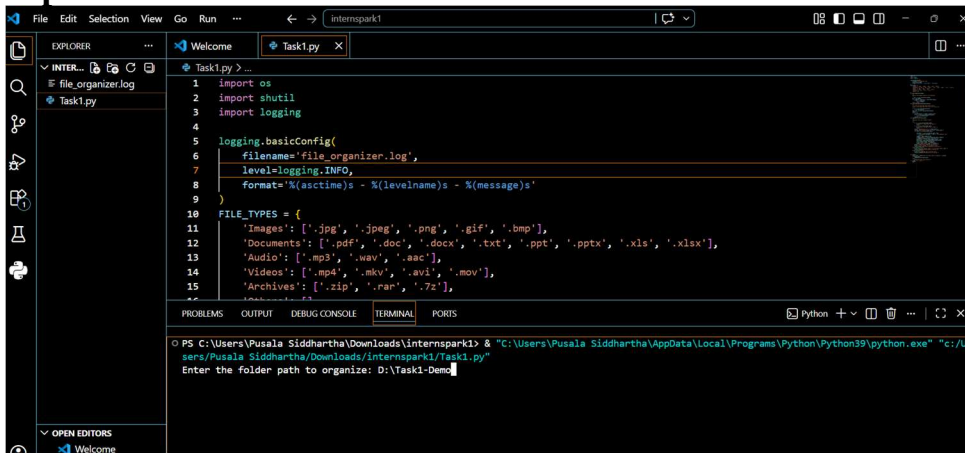
def main():
    folder_path = input("Enter the folder path to organize: ").strip()
    if folder_path:
        organize_files(folder_path)
    else:
        print("No folder path entered.")
if __name__ == "__main__":
    main()

```

Sample Input

Enter the folder path to organize: C:\Users\YourName\Downloads

Input Screenshot



Sample Output

Organizing files in: C:\Users\YourName\Downloads

Moved: photo.jpg -> Images

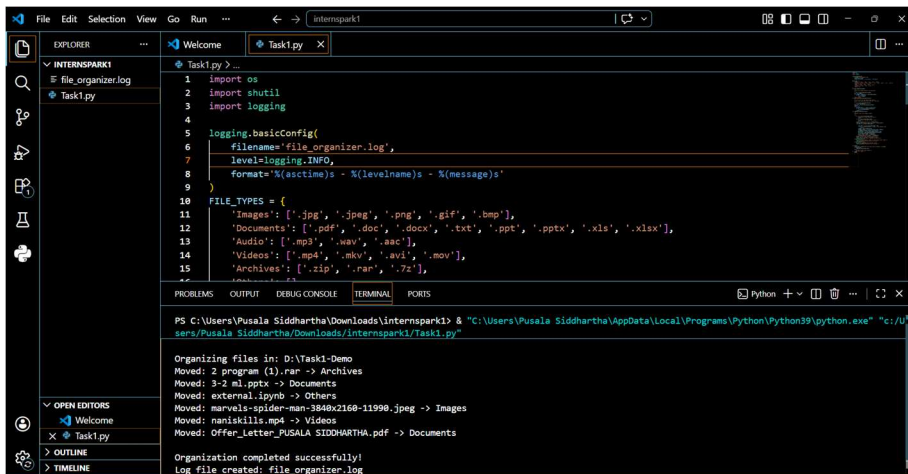
Moved: report.pdf -> Documents

Moved: song.mp3 -> Audio

Organization completed successfully!

Log file created: file_organizer.log

Output Screenshot

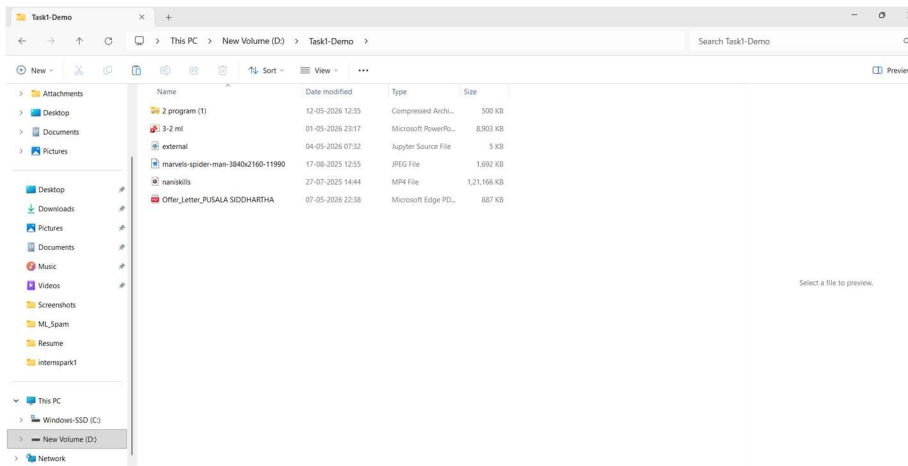


```
1 import os
2 import shutil
3 import logging
4
5 logging.basicConfig(
6     filename='file_organizer.log',
7     level=logging.INFO,
8     format='%(asctime)s - %(levelname)s - %(message)s'
9 )
10
11 FILE_TYPES = {
12     'Images': ['.jpg', '.jpeg', '.png', '.gif', '.bmp'],
13     'Documents': ['.pdf', '.doc', '.docx', '.txt', '.ppt', '.pptx', '.xls', '.xlsx'],
14     'Audio': ['.mp3', '.wav', '.aac'],
15     'Videos': ['.mp4', '.mkv', '.avi', '.mov'],
16     'Archives': ['.zip', '.rar', '.7z']
17 }
```

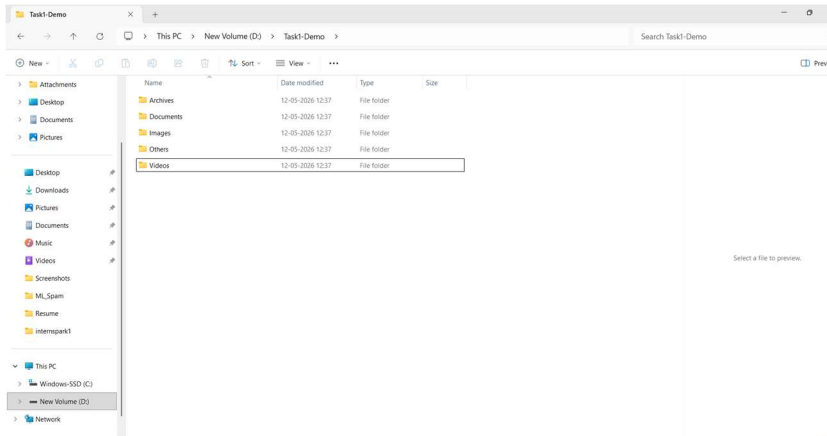
PS C:\Users\Pusala Siddhartha\Downloads\internspark1> "C:\Users\Pusala Siddhartha\AppData\Local\Programs\Python\Python39\python.exe" "c:/Users/Pusala Siddhartha/Downloads/internspark1/Task1.py"

Organizing files in: D:\Task1-Demo
Moved: 2 program (1).rar -> Archives
Moved: 3-2 ml.pptx -> Documents
Moved: external.ipynb -> Others
Moved: marvels-spider-man-3840x2160-11990.jpeg -> Images
Moved: naniskills.mp4 -> Videos
Moved: Offer_Letter_PUSALA SIDDHARTHA.pdf -> Documents
Organization completed successfully!
Log file created: file_organizer.log

Folder Before Execution



Folder After Execution



GitHub Repository Link

https://github.com/Siddharthapusala/Python_Automation_Script.git

Conclusion

The Python Automation Script successfully organizes files into categorized folders, handles duplicate names, creates logs, and uses user input and exception handling. All internship requirements are satisfied.